24.3.2 Program inspections

Program inspections are 'peer reviews' where team members collaborate to find bugs in the program that is being developed. As I discussed in Chapter 8, inspections may be part of the software verification and validation processes. They complement testing as they do not require the program to be executed. This means that incomplete versions of the system can be verified and that representations such as UML models can be checked. Gilb and Graham (1993) suggest that one of the most effective ways to use inspections is to review the test cases for a system. Inspections can discover problems with tests and so improve the effectiveness of these tests in detecting program bugs.

Program inspections involve team members from different backgrounds who make a careful, line-by-line review of the program source code. They look for defects and problems and describe these at an inspection meeting. Defects may be logical errors, anomalies in the code that might indicate an erroneous condition or features that have been omitted from the code. The review team examines the design models or the program code in detail and highlights anomalies and problems for repair.

During an inspection, a checklist of common programming errors is often used to focus the search for bugs. This checklist may be based on examples from books or from knowledge of defects that are common in a particular application domain. You use different checklists for different programming languages because each language has its own characteristic errors. Humphrey (1989), in a comprehensive discussion of inspections, gives a number of examples of inspection checklists.

Possible checks that might be made during the inspection process are shown in Figure 24.8. Gilb and Graham (1993) emphasize that each organization should develop its own inspection checklist based on local standards and practices. These checklists should be regularly updated, as new types of defects are found. The items in the checklist vary according to programming language because of the different levels of checking that are possible at compile-time. For example, a Java compiler checks that functions have the correct number of parameters; a C compiler does not.

Most companies that have introduced inspections have found that they are very effective in finding bugs. Fagan (1986) reported that more than 60 percent of the errors in a program can be detected using informal program inspections. Mills et al. (1987) suggest that a more formal approach to inspection, based on correctness arguments, can detect more than 90% of the errors in a program. McConnell (2004) compares unit testing, where the defect detection rate is about 25%, with inspections, where the defect detection rate was 60%. He also describes a number of case studies including an example where the introduction of peer reviews led to a 14% increase in productivity and a 90% decrease in program defects.

In spite of their well-publicized cost effectiveness, many software development companies are reluctant to use inspections or peer reviews. Software engineers with experience of program testing are sometimes unwilling to accept that inspections can

Fault class	Inspection check
Data faults	• Are all program variables initialized before their values are used? • Have all constants been named? • Should the upper bound of arrays be equal to the size of the array or Size -1? • If character strings are used, is a delimiter explicitly assigned? • Is there any possibility of buffer overflow?
Control faults	• For each conditional statement, is the condition correct? • Is each loop certain to terminate? • Are compound statements correctly bracketed? • In case statements, are all possible cases accounted for? • If a break is required after each case in case statements, has it been included?
Input/output faults	• Are all input variables used? • Are all output variables assigned a value before they are output? • Can unexpected inputs cause corruption?
Interface faults	• Do all function and method calls have the correct number of parameters? • Do formal and actual parameter types match? • Are the parameters in the right order? • If components access shared memory, do they have the same model of the shared memory structure?
Storage management faults	• If a linked structure is modified, have all links been correctly reassigned? • If dynamic storage is used, has space been allocated correctly? • Is space explicitly deallocated after it is no longer required?
Exception management faults	• Have all possible error conditions been taken into account?

Figure 24.8 An inspection checklist

be more effective for defect detection than testing. Managers may be suspicious because inspections require additional costs during design and development. They may not wish to take the risk that there will be no corresponding savings in program testing costs.

Agile processes rarely use formal inspection or peer review processes. Rather, they rely on team members cooperating to check each other's code, and informal guidelines, such as 'check before check-in', which suggest that programmers should check their own code. Extreme programming practitioners argue that pair programming is an effective substitute for inspection as this is, in effect, a continual inspection process. Two people look at every line of code and check it before it is accepted.

Pair programming leads to a deep knowledge of a program, as both programmers have to understand its working in detail to continue development. This depth of knowledge is sometimes difficult to achieve in other inspection processes and so pair programming can find bugs that sometimes would not be discovered in formal

inspections. However, pair programming can also lead to mutual misunderstandings of requirements, where both members of the pair make the same mistake. Furthermore, pairs may be reluctant to look for errors because the pair does not want to slow down the progress of the project. The people involved cannot be as objective as an external inspection team and their ability to discover defects is likely to be compromised by their close working relationship.

24.4 Software measurement and metrics

Software measurement is concerned with deriving a numeric value or profile for an attribute of a software component, system, or process. By comparing these values to each other and to the standards that apply across an organization, you may be able to draw conclusions about the quality of software, or assess the effectiveness of software processes, tools, and methods.

For example, say an organization intends to introduce a new software-testing tool. Before introducing the tool, you record the number of software defects discovered in a given time. This is a baseline for assessing the effectiveness of the tool. After using the tool for some time, you repeat this process. If more defects have been found in the same amount of time, after the tool has been introduced, then you may decide that it provides useful support for the software validation process.

The long-term goal of software measurement is to use measurement in place of reviews to make judgments about software quality. Using software measurement, a system could ideally be assessed using a range of metrics and, from these measurements, a value for the quality of the system could be inferred. If the software had reached a required quality threshold, then it could be approved without review. When appropriate, the measurement tools might also highlight areas of the software that could be improved. However, we are still quite a long way from this ideal situation and, there are no signs that automated quality assessment will become a reality in the foreseeable future.

A software metric is a characteristic of a software system, system documentation, or development process that can be objectively measured. Examples of metrics include the size of a product in lines of code; the Fog index (Gunning, 1962), which is a measure of the readability of a passage of written text; the number of reported faults in a delivered software product; and the number of person-days required to develop a system component.

Software metrics may be either control metrics or predictor metrics. As the names imply, control metrics support process management, and predictor metrics help you predict characteristics of the software. Control metrics are usually associated with software processes. Examples of control or process metrics are the average effort and the time required to repair reported defects. Predictor metrics are associated with the software itself and are sometimes known as ‘product metrics’. Examples of predictor metrics are the cyclomatic complexity of a module (discussed in Chapter 8),

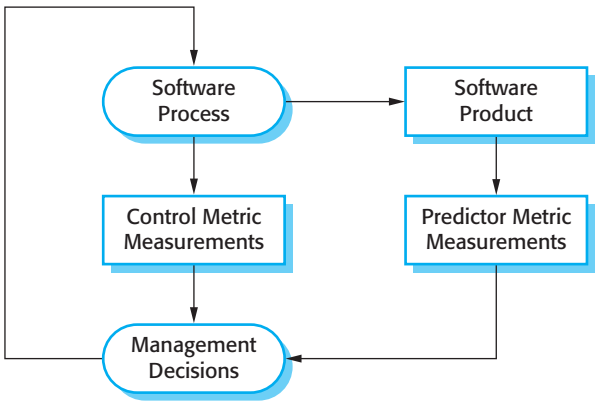


Figure 24.9 Predictor and control measurements

the average length of identifiers in a program, and the number of attributes and operations associated with object classes in a design.

Both control and predictor metrics may influence management decision making, as shown in Figure 24.9. Managers use process measurements to decide if process changes should be made, and predictor metrics to help estimate the effort required to make software changes. In this chapter, I mostly discuss predictor metrics, whose values are assessed by analyzing the code of a software system. I discuss control metrics and how they are used in process improvement in Chapter 26.

There are two ways in which measurements of a software system may be used:

1. *To assign a value to system quality attributes* By measuring the characteristics of system components, such as their cyclomatic complexity, and then aggregating these measurements, you can assess system quality attributes, such as maintainability.

2. *To identify the system components whose quality is substandard* Measurements can identify individual components with characteristics that deviate from the norm. For example, you can measure components to discover those with the highest complexity. These are most likely to contain bugs because the complexity makes them harder to understand.

Unfortunately, it is difficult to make direct measurements of many of the software quality attributes shown in Figure 24.2. Quality attributes such as maintainability, understandability, and usability are external attributes that relate to how developers and users experience the software. They are affected by subjective factors, such as user experience and education, and they cannot therefore be measured objectively. To make a judgment about these attributes, you have to measure some internal attributes of the software (such as its size, complexity, etc.) and assume that these are related to the quality characteristics that you are concerned with.

Figure 24.10 shows some external software quality attributes and internal attributes that could, intuitively, be related to them. The diagram suggests that there may be relationships between external and internal attributes, but it does not say how

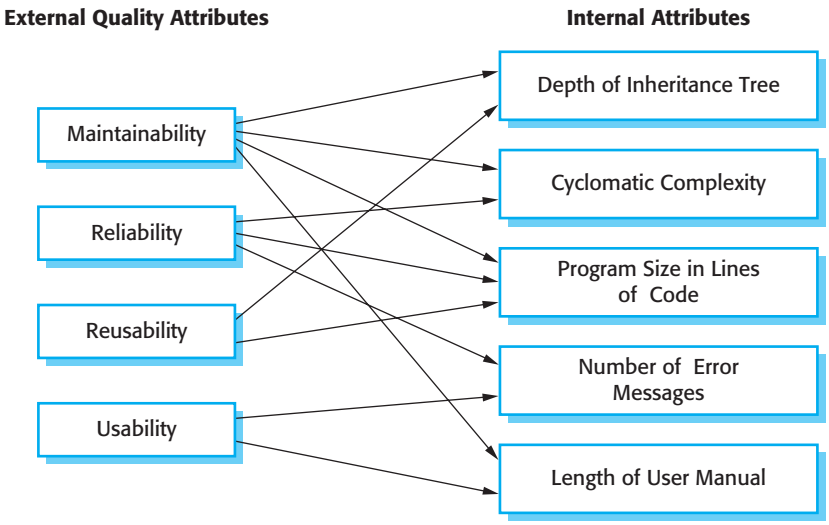


Figure 24.10 Relationships between internal and external software

these attributes are related. If the measure of the internal attribute is to be a useful predictor of the external software characteristic, three conditions must hold (Kitchenham, 1990):

1. The internal attribute must be measured accurately. This is not always straightforward and it may require special-purpose tools to make the measurements.
2. A relationship must exist between the attribute that can be measured and the external quality attribute that is of interest. That is, the value of the quality attribute must be related, in some way, to the value of the attribute than can be measured.
3. This relationship between the internal and external attributes must be understood, validated, and expressed in terms of a formula or model. Model formulation involves identifying the functional form of the model (linear, exponential, etc.) by analysis of collected data, identifying the parameters that are to be included in the model, and calibrating these parameters using existing data.

Internal software attributes, such as the cyclomatic complexity of a component, are measured by using software tools that analyze the source code of the software. Open source tools are available that can be used to make these measurements. Although intuition suggests that there could be a relationship between the complexity of a software component and the number of observed failures in use, it is difficult to objectively demonstrate that this is the case. To test this hypothesis, you need failure data for a large number of components and access to the component source code for analysis. Very few companies have made a long-term commitment to collecting data about their software, so failure data for analysis is rarely available.

In the 1990s, several large companies such as Hewlett-Packard (Grady, 1993), AT&T (Barnard and Price, 1994), and Nokia (Kilpi, 2001) introduced metrics

programs. They made measurements of their products and processes and used these in their quality management processes. Most of the focus was on collecting metrics on program defects and the verification and validation processes. Offen and Jeffrey (1997) and Hall and Fenton (1997) discuss the introduction of metrics programs in industry in more detail.

There is little information publicly available about the current use of systematic software measurement in industry. Many companies do collect information about their software, such as the number of requirements change requests or the number of defects discovered in testing. However, it is not clear if they then use these measurements systematically to compare software products and processes or assess the impact of changes to software processes and tools. There are several reasons why this is difficult:

1. It is impossible to quantify the return on investment of introducing an organizational metrics program. There have been significant improvements in software quality over the past few years without the use of metrics so it is difficult to justify the initial costs of introducing systematic software measurement and assessment.
2. There are no standards for software metrics or standardized processes for measurement and analysis. Many companies are reluctant to introduce measurement programs until such standards and supporting tools are available.
3. In many companies, software processes are not standardized and are poorly defined and controlled. As such, there is too much process variability within the same company for measurements to be used in a meaningful way.
4. Much of the research on software measurement and metrics has focused on code-based metrics and plan-driven development processes. However, more and more software is now developed by configuring ERP systems or COTS, or by using agile methods. We don't know, therefore, if previous research is applicable to these software development techniques.
5. Introducing measurement adds additional overhead to processes. This contradicts the aims of agile methods, which recommend the elimination of process activities that are not directly related to program development. Companies that have adopted agile methods are therefore not likely to adopt a metrics program.

Software measurement and metrics are the basis of empirical software engineering (Endres and Rombach, 2003). This is a research area in which experiments on software systems and the collection of data about real projects has been used to form and validate hypotheses about software engineering methods and techniques. Researchers working in this area argue that we can only be confident of the value of software engineering methods and techniques if we can provide concrete evidence that they actually provide the benefits that their inventors suggest.

Unfortunately, even when it is possible to make objective measurements and draw conclusions from them, these will not necessarily convince decision makers. Rather, decision making is often influenced by subjective factors, such as novelty, or the

extent to which techniques are of interest to practitioners. I think, therefore, that it will be many years before the results from empirical software engineering have a significant effect on software engineering practice.

24.4.1 Product metrics

Product metrics are predictor metrics that are used to measure internal attributes of a software system. Examples of product metrics include the system size, measured in lines of code, or the number of methods associated with each object class. Unfortunately, as I have explained earlier in this section, software characteristics that can be easily measured, such as size and cyclomatic complexity, do not have a clear and consistent relationship with quality attributes such as understandability and maintainability. The relationships vary depending on the development processes and technology used and the type of system that is being developed.

Product metrics fall into two classes:

1. Dynamic metrics, which are collected by measurements made of a program in execution. These metrics can be collected during system testing or after the system has gone into use. An example might be the number of bug reports or the time taken to complete a computation.
2. Static metrics, which are collected by measurements made of representations of the system, such as the design, program, or documentation. Examples of static metrics are the code size and the average length of identifiers used.

These types of metric are related to different quality attributes. Dynamic metrics help to assess the efficiency and reliability of a program. Static metrics help assess the complexity, understandability, and maintainability of a software system or system components.

There is usually a clear relationship between dynamic metrics and software quality characteristics. It is fairly easy to measure the execution time required for particular functions and to assess the time required to start up a system. These relate directly to the system's efficiency. Similarly, the number of system failures and the type of failure can be logged and related directly to the reliability of the software, as discussed in Chapter 15.

As I have discussed, static metrics, such as those shown in Figure 24.11, have an indirect relationship with quality attributes. A large number of different metrics have been proposed and many experiments have tried to derive and validate the relationships between these metrics and attributes such as system complexity and maintainability. None of these experiments have been conclusive but program size and control complexity seem to be the most reliable predictors of understandability, system complexity, and maintainability.

The metrics in Figure 24.11 are applicable to any program but more specific object-oriented (OO) metrics have also been proposed. Figure 24.12 summarizes

Software metric	Description
Fan-in/Fan-out	Fan-in is a measure of the number of functions or methods that call another function or method (say X). Fan-out is the number of functions that are called by function X. A high value for fan-in means that X is tightly coupled to the rest of the design and changes to X will have extensive knock-on effects. A high value for fan-out suggests that the overall complexity of X may be high because of the complexity of the control logic needed to coordinate the called components.
Length of code	This is a measure of the size of a program. Generally, the larger the size of the code of a component, the more complex and error-prone that component is likely to be. Length of code has been shown to be one of the most reliable metrics for predicting error-proneness in components.
Cyclomatic complexity	This is a measure of the control complexity of a program. This control complexity may be related to program understandability. I discuss cyclomatic complexity in Chapter 8.
Length of identifiers	This is a measure of the average length of identifiers (names for variables, classes, methods, etc.) in a program. The longer the identifiers, the more likely they are to be meaningful and hence the more understandable the program.
Depth of conditional nesting	This is a measure of the depth of nesting of if-statements in a program. Deeply nested if-statements are hard to understand and potentially error-prone.
Fog index	This is a measure of the average length of words and sentences in documents. The higher the value of a document's Fog index, the more difficult the document is to understand.

Figure 24.11 Static software product metrics

Chidamber and Kemerer's suite (sometimes called the CK suite) of six object-oriented metrics (1994). Although these were originally proposed in the early 1990s, they are still the most widely used OO metrics. Some UML design tools automatically collect values for these metrics as UML diagrams are created.

El-Amam (2001), in an excellent review of object-oriented metrics, discusses the CK metrics and other OO metrics, and concludes that we do not yet have sufficient evidence to understand how these and other object-oriented metrics relate to external software qualities. This situation has not really changed since his analysis in 2001. We still don't know how to use measurements of object-oriented programs to draw reliable conclusions about their quality.

24.4.2 Software component analysis

A measurement process that may be part of a software quality assessment process is shown in Figure 24.13. Each system component can be analyzed separately using a range of metrics. The values of these metrics may then be compared for different

Object-oriented metric	Description
Weighted methods per class (WMC)	This is the number of methods in each class, weighted by the complexity of each method. Therefore, a simple method may have a complexity of 1, and a large and complex method a much higher value. The larger the value for this metric, the more complex the object class. Complex objects are more likely to be difficult to understand. They may not be logically cohesive, so cannot be reused effectively as superclasses in an inheritance tree.
Depth of inheritance tree (DIT)	This represents the number of discrete levels in the inheritance tree where subclasses inherit attributes and operations (methods) from superclasses. The deeper the inheritance tree, the more complex the design. Many object classes may have to be understood to understand the object classes at the leaves of the tree.
Number of children (NOC)	This is a measure of the number of immediate subclasses in a class. It measures the breadth of a class hierarchy, whereas DIT measures its depth. A high value for NOC may indicate greater reuse. It may mean that more effort should be made in validating base classes because of the number of subclasses that depend on them.
Coupling between object classes (CBO)	Classes are coupled when methods in one class use methods or instance variables defined in a different class. CBO is a measure of how much coupling exists. A high value for CBO means that classes are highly dependent, and therefore it is more likely that changing one class will affect other classes in the program.
Response for a class (RFC)	RFC is a measure of the number of methods that could potentially be executed in response to a message received by an object of that class. Again, RFC is related to complexity. The higher the value for RFC, the more complex a class and hence the more likely it is that it will include errors.
Lack of cohesion in methods (LCOM)	LCOM is calculated by considering pairs of methods in a class. LCOM is the difference between the number of method pairs without shared attributes and the number of method pairs with shared attributes. The value of this metric has been widely debated and it exists in several variations. It is not clear if it really adds any additional, useful information over and above that provided by other metrics.

Figure 24.12 The CK object-oriented metrics suite

components and, perhaps, with historical measurement data collected on previous projects. Anomalous measurements, which deviate significantly from the norm, may imply that there are problems with the quality of these components.

The key stages in this component measurement process are:

1. *Choose measurements to be made* The questions that the measurement is intended to answer should be formulated and the measurements required to answer these questions defined. Measurements that are not directly relevant to these questions need not be collected. Basili's GQM (Goal-Question-Metric) paradigm (Basili and Rombach, 1988), discussed in Chapter 26, is a good approach to use when deciding what data is to be collected.

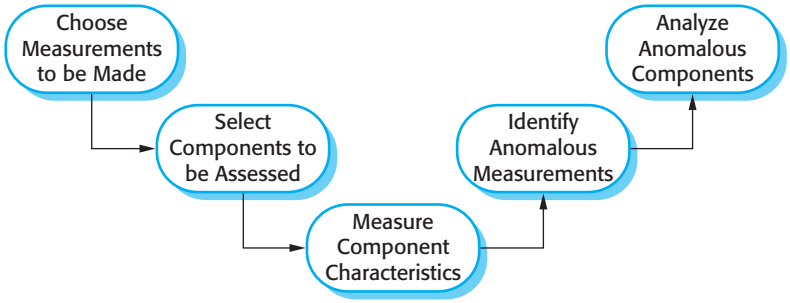


Figure 24.13 The process of product measurement

2. *Select components to be assessed* You may not need to assess metric values for all of the components in a software system. Sometimes, you can select a representative selection of components for measurement, allowing you to make an overall assessment of system quality. At other times, you may wish to focus on the core components of the system that are in almost constant use. The quality of these components is more important than the quality of components that are only rarely used.

3. *Measure component characteristics* The selected components are measured and the associated metric values computed. This normally involves processing the component representation (design, code, etc.) using an automated data collection tool. This tool may be specially written or may be a feature of design tools that are already in use.

4. *Identify anomalous measurements* After the component measurements have been made, you then compare them with each other and to previous measurements that have been recorded in a measurement database. You should look for unusually high or low values for each metric, as these suggest that there could be problems with the component exhibiting these values.

5. *Analyze anomalous components* When you have identified components that have anomalous values for your chosen metrics, you should examine them to decide whether or not these anomalous metric values mean that the quality of the component is compromised. An anomalous metric value for complexity (say) does not necessarily mean a poor quality component. There may be some other reason for the high value, so may not mean that there are component quality problems.

You should always maintain collected data as an organizational resource and keep historical records of all projects even when data has not been used during a particular project. Once a sufficiently large measurement database has been established, you can then make comparisons of software quality across projects and validate the relations between internal component attributes and quality characteristics.

24.4.3 Measurement ambiguity

When you collect quantitative data about software and software processes, you have to analyze that data to understand its meaning. It is easy to misinterpret data and to make inferences that are incorrect. You cannot simply look at the data on its own—you must also consider the context where the data is collected.

To illustrate how collected data can be interpreted in different ways, consider the scenario below, which is concerned with the number of change requests made by users of a system:

A manager decides to monitor the number of change requests submitted by customers based on an assumption that there is a relationship between these change requests and product usability and suitability. She assumes that the higher the number of change requests, the less the software meets the needs of the customer.

Handling change requests and changing the software is expensive. The organization therefore decides to modify its process with the aim of improving customer satisfaction and, at the same time, reduce the costs of making changes. The intent is that the process changes will result in better products and fewer change requests.

Process changes are initiated to increase customer involvement in the software design process. Beta testing of all products is introduced and customer-requested modifications are incorporated in the delivered product. New versions of products, developed with this modified process, are delivered. In some cases, the number of change requests is reduced. In others, it is increased. The manager is baffled and finds it impossible to assess the effects of the process changes on the product quality.

To understand why this kind of ambiguity can occur, you have to understand the reasons why users might make change requests:

1. The software is not good enough and does not do what customers want it to do. They therefore request changes to deliver the functionality that they require.
2. Alternatively, the software may be very good and so it is widely and heavily used. Change requests may be generated because there are many software users who creatively think of new things that could be done with the software.

Therefore, increasing the customer involvement in the process may reduce the number of change requests for products where the customers were unhappy. The process changes have been effective and have made the software more usable and suitable. Alternatively, however, the process changes may not have worked and customers may have decided to look for an alternative system. The number of change requests might decrease because the product has lost market share to a rival product and there are consequently fewer product users.

On the other hand, the process changes might lead to many new, happy customers who wish to participate in the product development process. They therefore generate

more change requests. Changes to the process of handling change requests may contribute to this increase. If the company is more responsive to customers, they may generate more change requests because they know that these requests will be taken seriously. They believe that their suggestions will probably be incorporated in later versions of the software. Alternatively, the number of change requests might have increased because the beta-test sites were not typical of most usage of the program.

To analyze the change request data, you do not simply need to know the number of change requests. You need to know who made the request, how they use the software, and why the request was made. You also need information about external factors such as modifications to the change request procedure or market changes that might have an effect. With this information, it is then possible to find out if the process changes have been effective in increasing product quality.

This illustrates the difficulties of understanding the effects of changes and the 'scientific' approach to this problem is to reduce the number of factors that might affect the measurements made. However, processes and products that are being measured are not insulated from their environment. The business environment is constantly changing and it is impossible to avoid changes to work practice just because they may make comparisons of data invalid. As such, quantitative data about human activities cannot always be taken at face value. The reasons why a measured value changes are often ambiguous. These reasons must be investigated in detail before drawing conclusions from any measurements that have been made.

KEY POINTS

- Software quality management is concerned with ensuring that software has a low number of defects and that it reaches the required standards of maintainability, reliability, portability, and so on. It includes defining standards for processes and products and establishing processes to check that these standards have been followed.
- Software standards are important for quality assurance as they represent an identification of 'best practice'. When developing software, standards provide a solid foundation for building good quality software.
- You should document a set of quality assurance procedures in an organizational quality manual. This may be based on the generic model for a quality manual suggested in the ISO 9001 standard.
- Reviews of the software process deliverables involve a team of people who check that quality standards are being followed. Reviews are the most widely used technique for assessing quality.
- In a program inspection or peer review, a small team systematically checks the code. They read the code in detail and look for possible errors and omissions. The problems detected are then discussed at a code review meeting.

- Software measurement can be used to gather quantitative data about software and the software process. You may be able to use the values of the software metrics that are collected to make inferences about product and process quality.
- Product quality metrics are particularly useful for highlighting anomalous components that may have quality problems. These components should then be analyzed in more detail.

FURTHER READING

Metrics and Models for Software Quality Engineering, 2nd edition. This is a very comprehensive discussion of software metrics covering process-, product-, and object-oriented metrics. It also includes some background on the mathematics required to develop and understand models based on software measurement. (S. H. Kan, Addison-Wesley, 2003.)

Software Quality Assurance: From Theory to Implementation. An excellent, up-to-date look at the principles and practice of software quality assurance. It includes a discussion of standards such as ISO 9001. (D. Galin, Addison-Wesley, 2004.)

'A Practical Approach for Quality-Driven Inspections'. Many articles on inspections are now rather old and do not take modern software development practice into account. This relatively recent article describes an inspection method that addresses some of the problems of using inspection and suggests how inspection can be used in a modern development environment. (C. Denger, F. Shull, *IEEE Software*, **24** (2), March–April 2007.) http://dx.doi.org/10.1109/MS.2007.31

'Misleading Metrics and Unsound Analyses'. An excellent article by leading metrics researchers that discusses the difficulties of understanding what measurements really mean. (B. Kitchenham, R. Jeffrey and C. Connaughton, *IEEE Software*, **24** (2), March–April 2007.) http://dx.doi.org/10.1109/MS.2007.49.

'The Case for Quantitative Project Management'. This is an introduction to a special section in the magazine that includes two other articles on quantitative project management. It makes an argument for further research in metrics and measurement to improve software project management. (B. Curtis et al., *IEEE Software*, **25** (3), May–June 2008.) http://dx.doi.org/10.1109/MS.2008.80.

EXERCISES

24.1. Explain why a high-quality software process should lead to high-quality software products. Discuss possible problems with this system of quality management.

24.2. Explain how standards may be used to capture organizational wisdom about effective methods of software development. Suggest four types of knowledge that might be captured in organizational standards.

24.3. Discuss the assessment of software quality according to the quality attributes shown in Figure 24.2. You should consider each attribute in turn and explain how it might be assessed.

24.4. Design an electronic form that may be used to record review comments and which could be used to electronically mail comments to reviewers.

24.5. Briefly describe possible standards that might be used for:

- The use of control constructs in C, C#, or Java;
- Reports which might be submitted for a term project in a university;
- The process of making and approving program changes (see Chapter 26);
- The process of purchasing and installing a new computer.

24.6. Assume you work for an organization that develops database products for individuals and small businesses. This organization is interested in quantifying its software development. Write a report suggesting appropriate metrics and suggest how these can be collected.

24.7. Explain why program inspections are an effective technique for discovering errors in a program. What types of error are unlikely to be discovered through inspections?

24.8. Explain why design metrics are, by themselves, an inadequate method of predicting design quality.

24.9. Explain why it is difficult to validate the relationships between internal product attributes, such as cyclomatic complexity and external attributes, such as maintainability.

24.10. A colleague who is a very good programmer produces software with a low number of defects but consistently ignores organizational quality standards. How should her managers react to this behavior?

REFERENCES

Bamford, R. and Deibler, W. J. (eds.) (2003). 'ISO 9001:2000 for Software and Systems Providers: An Engineering Approach'. Boca Raton, Fla.: CRC Press.

Barnard, J. and Price, A. (1994). 'Managing Code Inspection Information'. *IEEE Software*, **11** (2), 59–69.

Basili, V. R. and Rombach, H. D. (1988). 'The TAME project: Towards Improvement-Oriented Software Environments'. *IEEE Trans. on Software Eng.*, **14** (6), 758–773.

Boehm, B. W., Brown, J. R., Kaspar, H., Lipow, M., Macleod, G. and Merrit, M. (1978). *Characteristics of Software Quality*. Amsterdam: North-Holland.

Chidamber, S. and Kemerer, C. (1994). 'A Metrics Suite for Object-Oriented Design'. *IEEE Trans. on Software Eng.*, **20** (6), 476–93.

Crosby, P. (1979). *Quality is Free*. New York: McGraw-Hill.

El-Amam, K. 2001. 'Object-oriented Metrics: A Review of Theory and Practice'. National Research Council of Canada. http://seg.iit.nrc.ca/English/abstracts/NRC44190.html .

Endres, A. and Rombach, D. (2003). *Empirical Software Engineering: A Handbook of Observations, Laws and Theories*. Harlow, UK: Addison-Wesley.

Fagan, M. E. (1976). 'Design and code inspections to reduce errors in program development'. *IBM Systems J.*, **15** (3), 182–211.

Fagan, M. E. (1986). 'Advances in Software Inspections'. *IEEE Trans. on Software Eng.*, **SE-12** (7), 744–51.

Gilb, T. and Graham, D. (1993). *Software Inspection*. Wokingham: Addison-Wesley.

Grady, R. B. (1993). 'Practical Results from Measuring Software Quality'. *Comm. ACM*, **36** (11), 62–8.

Gunning, R. (1962). *Techniques of Clear Writing*. New York: McGraw-Hill.

Hall, T. and Fenton, N. (1997). 'Implementing Effective Software Metrics Programs'. *IEEE Software*, **14** (2), 55–64.

Humphrey, W. (1989). *Managing the Software Process*. Reading, Mass.: Addison-Wesley.

IEC. 1998. 'Standard IEC 61508: Functional safety of electrical/electronic/programmable electronic safety-related systems'. International Electrotechnical Commission: Geneva.

IEEE. (2003). *IEEE Software Engineering Standards Collection on CD-ROM*. Los Alamitos, Ca.: IEEE Computer Society Press.

Ince, D. (1994). *ISO 9001 and Software Quality Assurance*. London: McGraw-Hill.

Kilpi, T. (2001). 'Implementing a Software Metrics Program at Nokia'. *IEEE Software*, **18** (6), 72–7.

Kitchenham, B. (1990). 'Measuring Software Development'. In *Software Reliability Handbook*. Rook, P. (ed.). Amsterdam: Elsevier, 303–31.

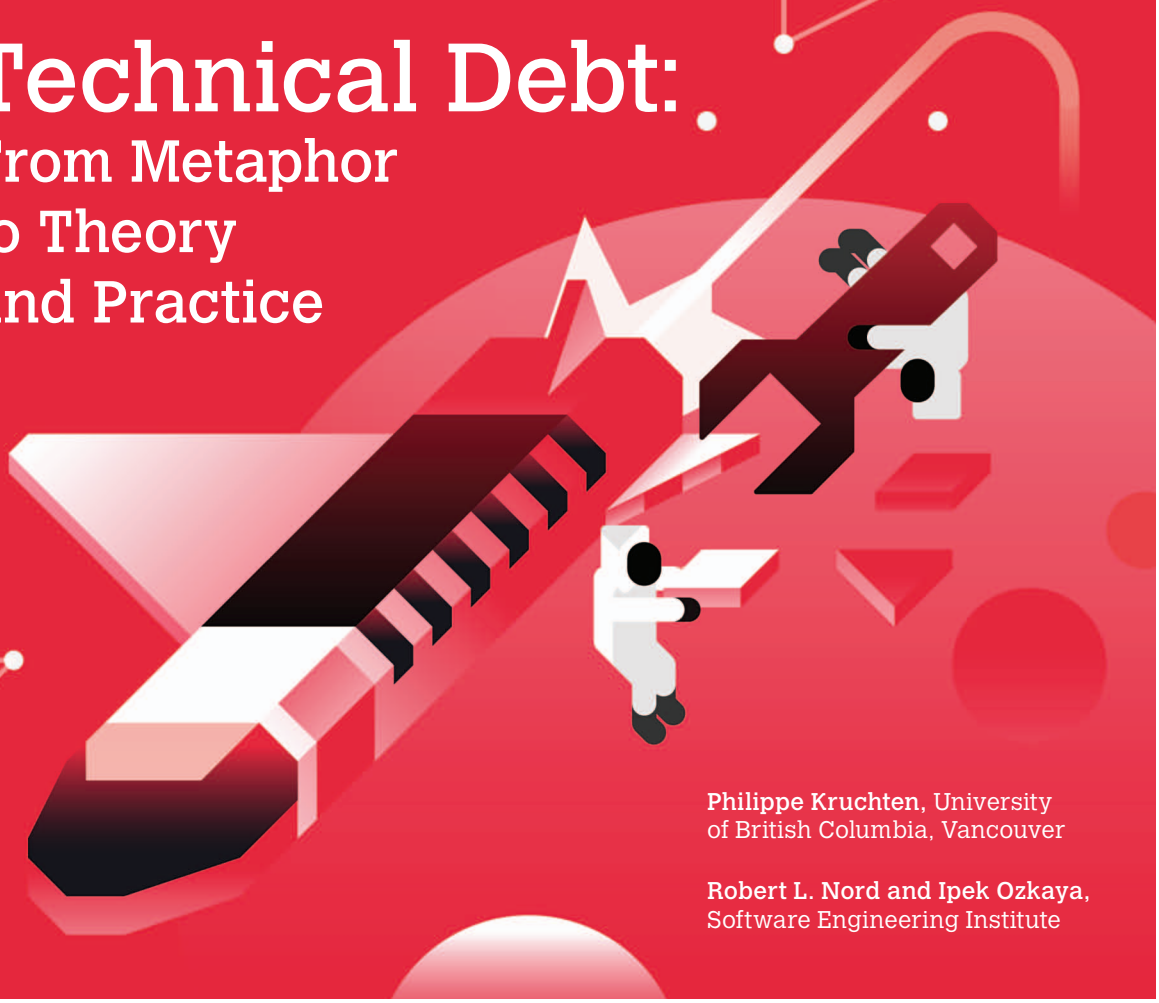
McConnell, S. (2004). *Code Complete: A Practical Handbook of Software Construction, 2nd edition*. Seattle: Microsoft Press.

Mills, H. D., Dyer, M. and Linger, R. (1987). 'Cleanroom Software Engineering'. *IEEE Software*, **4** (5), 19–25.

Offen, R. J. and Jeffrey, R. (1997). 'Establishing Software Measurement Programs'. *IEEE Software*, **14** (2), 45–54.

Stalhane, T. and Hanssen, G. K. (2008). 'The application of ISO 9001 to agile software development'. *9th International Conference on Product Focused Software Process Improvement, PROFES 2008*, Monte Porzio Catone, Italy: Springer.

Technical Debt: From Metaphor to Theory and Practice



Philippe Kruchten, University of British Columbia, Vancouver

Robert L. Nord and Ipek Ozkaya, Software Engineering Institute

THE METAPHOR OF technical debt in software development was introduced two decades ago by Ward Cunningham[1] to explain to nontechnical product stakeholders the need for what we call now "refactoring." It has been refined and expanded since, notably by Steve McConnell in his taxonomy,[2] Martin Fowler with his four quadrants,[3] and Jim Highsmith and his colleagues from the Cutter Consortium with their model of the impact of technical debt on the total cost of ownership.[4]

See www.computer.org/software-multimedia for multimedia content related to this article.

From the original description—"not quite right code which we postpone making it right"[1]—various people have used the metaphor of technical "debt" to describe many other kinds of debts or ills of software development, encompassing broadly anything that stands in the way of deploying, selling, or evolving a software system or anything that adds to the friction from which software development endeavors suffer: test debt, people debt, architectural debt, requirement debt, documentation debt, or just an amorphous, all-encompassing software debt.[5] Consequently, the concept of technical debt in software development has become somewhat diluted lately. Is a new requirement, function, or feature not yet implemented "requirement debt"? Do we call postponing the development of a new function "planning debt"? The metaphor is losing some of its strength.

Furthermore, once we identify tools such as static code analyzers to assist us in identifying technical debt, there's a danger of equating it with whatever our tools can detect. This approach leads to leaving aside large amounts of potential technical debt that's undetectable by tools, such as structural or architectural debt or technological gaps. Gaps in technology are of particular interest because the debt incurred

0740-7459/12/$31.00 © 2012 IEEE

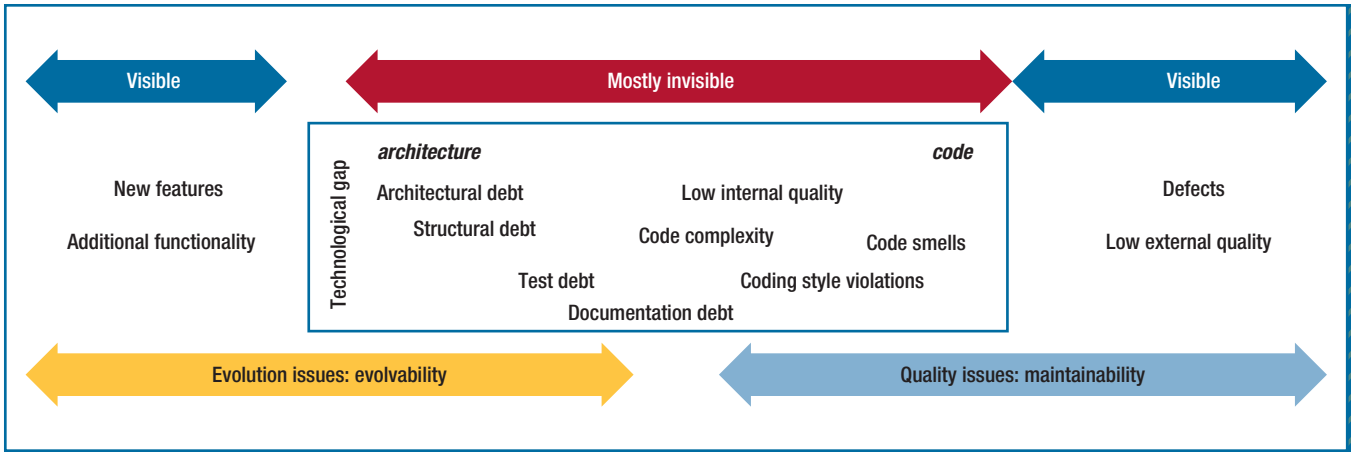


FIGURE 1. The technical debt landscape. On the left, evolution or its challenges; on the right, quality issues, both internal and external.

isn't the result of having made a wrong choice originally, but rather the result of the context's evolution—the passing of time—so that the choice isn't quite right in retrospect. Technical debt in this case is due to external events: technological obsolescence, change of environment, rapid commercial success, advent of new and better technologies, and so on—in other words, the invisible aspects of natural software aging and evolution. You could even argue that "gold plating" an architectural design, making the system more flexible and adaptable than it actually needs to be, can be a form of technical debt, if this added flexibility hinders future development without actually being exploited.

Organizing the Technical Debt Landscape

To make some progress, we need to go beyond debt as a "rhetorical concept."[6] We need a better definition of what constitutes technical debt and some perspective or viewpoints that let us reason across a wide range of technical debt. In short, we need a theoretical foundation.

Figure 1 shows a possible organization of a technical debt landscape—or rather, of software improvement from a given state. We can distinguish visible elements such as new functionality to add and defects to fix, and the invisible elements (or rather, those visible only to software developers). We can see that on the left, we're dealing primarily with evolution or its challenges, whereas on the right, we're dealing with quality issues, both internal and external. We propose to limit debt to the invisible elements—that is, to the elements in the rectangular box, including the invisible aspects of evolution and quality.

Tackling Technical Debt

Most authors agree that the major cause of technical debt is schedule pressure. However, on the right side of the picture, when debt is associated with quality and maintainability issues, other causes become probable, such as carelessness, lack of education, poor processes, nonsystematic verification of quality, or basic incompetence.

Because they use an iterative development process, many agile teams seem to believe that they're completely immune to technical debt. Although iterations offer the opportunity to reimburse debt in a timely fashion, the opposite often occurs. Developing and delivering very rapidly, with no time for proper design or to reflect on the longer term, and a lack of rigor or systematic testing (including automated testing) leads some agile projects into massive amounts of debt very rapidly. In fact, such debt can mount much more quickly than in any old-fashioned waterfall-like project. But in the end, it's all a matter of choice: where time to market is essential, the debt might actually be a good investment, but it's imperative to remain aware of this debt and the increased friction it will impose on the development team, as Cunningham suggested.[1]

So how can we tackle technical debt, or at least avoid accumulating too much of it? The first step is awareness: identifying debt and its causes. The next step is to manage this debt explicitly, which involves listing debt-related tasks in a common backlog during release and iteration planning, along with other "things to do."[7] Figure 2 illustrates how these elements might be organized in a backlog.[8] The element areas' colors reconcile four types of possible improvements—the tasks to attend to in the future to increase value, such as adding new features (green) or investing in the architecture (yellow), and to reduce the negative effects on value of defects (red) or technical debt (black).

Project backlogs often contain only

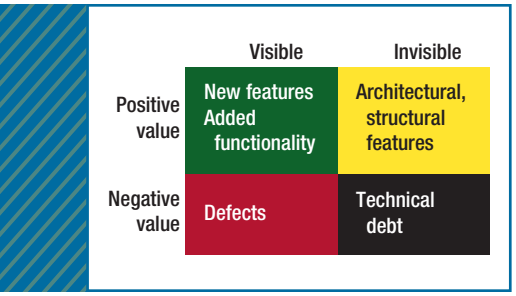


FIGURE 2. Four colors in a backlog. The element areas reconcile four types of possible improvements—the tasks to attend to in the future to increase value, such as adding new features (green) or investing in the architecture (yellow), and to reduce the negative effects on value of defects (red) or technical debt (black).

the green elements; a few technical practitioners keep in mind the yellow elements. The red elements appear elsewhere, perhaps in a defect database, and the black elements are nowhere to be found but they increasingly cripple the development, reducing velocity.

It's important to keep in mind, however, that technical debt is not only about code and code quality. Code analysis tools will identify a small number of the black elements. Therefore, code analysis tools aren't sufficient for identifying technical debt: more often than not, technical debt isn't related to code and its intrinsic qualities but to structural or architectural choices or to technological gaps. No tool will reveal that, two years ago, the team should have used some tool to internationalize and localize the code.

Architecture plays a significant role in the development of large systems, together with other development activities, such as documentation and testing (which are often lacking). These activities can add significantly to the debt and thus are part of the technical debt landscape in Figure 1. Code analysis will only tackle the right side of the box. Professionalism, diligence, dedication, and craftsmanship will certainly help, but they aren't the key determinants in reducing technical debt.

A Unified Theory?

Kevin Sullivan suggested that a simple model for tackling technical debt represents a software development endeavor as a sequence of changes, most of them improvements.[9] At a given point in time, the past set of changes is what defines the current state of the software. Some of these past changes are the events that triggered the current debt: the change or the way it was implemented isn't quite right from the current perspective.

The main issue facing the software development organization is how to decide about future changes: What evolution should the software system undergo, and in which sequence? This evolution is, in most cases, constrained by cost: the resources available to apply to making these changes, most likely driven by value, as viewed by external stakeholders.

The decision-making process about which sequence of changes to apply could be the main reconciling point across the whole landscape shown in Figure 1, from adding new features and adapting to new technologies to fixing defects and improving the quality, intrinsic or extrinsic. Because this decision process is about balancing cost and value, perhaps economic or financial models could become the unifying concept behind the whole landscape. A few have already been explored to some degree:

- Net Present Value (NPV) for a product, from the finance world;
- opportunity cost;
- real option analysis (ROA), or valuation; and
- total cost of ownership (TCO) for an IT system.

These four models were discussed in a recent ICSE workshop on technical debt,[9] with one of them (NPV) offering the most promise: it's better formalized than opportunity cost and simpler and less proprietary than TCO, while ROA can be seen as a probabilistic extension to NPV. TCO presents the danger, mentioned earlier, of diluting technical debt by introducing elements not related to software development (deployment, operations, and support).

Technical debt shouldn't be treated in isolation from adding new functionality or fixing defects, even though these aren't included in the definition of debt presented here. The challenge is in expressing all software development activities in terms of sequences of changes associated with a cost and a value (over time). These changes aren't independent, unfortunately. Their interdependencies play a big role—as Mark Denne and Jane Cleland-Huang have shown, in particular, visible features depend on less visible architectural aspects.[10]

In this new perspective, a system's technical debt at a given point in time could be defined as deferred investment opportunities or poorly managed risks.

In This Issue

This installment of *IEEE Software* gives readers different illustrations of the multifaceted concept of technical debt. Erin Lim, Nitin Taksante, and Carolyn Seaman went out into industry to check how software developers actually conceptualize, perceive, experience, and manage technical debt. They report their results in "A Balancing Act: What Software Practitioners Have to Say about Technical Debt." Their analysis describes the large and complex trade space of stakeholders' short- and long-term concerns and their strategies to keep them in balance.

Raja Bavani complements this view from the trenches with interviews of two agile experts, Johanna Rothman and Lisa Crispin, in "Distributed

Teams, Agile Testing, and Technical Debt." He then goes on to offer his own taxonomy of technical debt and how it relates to testing.

Bill Curtis, Jay Sappidi, and Alexandra Szynkarski explore the viability of an estimation framework for detecting technical debt using real-world data. They use the code analysis toolkit developed by CAST Software to identify technical debt in large systems, based on structural quality data, and literally put a price on it in "Estimating the Principal of an Application's Technical Debt."

As an alternative, Jean-Louis Letouzey and Michel Ilkiewicz describe "Managing Technical Debt with the SQALE Method." The SQALE approach is based on an analysis of an application's source code, using the indicators of quality attributes defined by the ISO systems and software quality standard (testability, maintainability, portability, and so on) to narrow down the point of focus.

Have we outgrown the financial debt metaphor? Does it still work? Do we misuse it? Israel Gat and Christof Ebert disagree on this topic in a point/counterpoint article.

We hope to keep this debt metaphor useful by confining it to what is really a debt—namely, the invisible result of past decisions about software that negatively affect its future—and by not extending the concept to anything that has a cost. From a practical perspective, we hope to see more tools and methods to identify and manage debt, covering more elements of the landscape. From a theoretical standpoint, we'll see models emerging, very likely rooted in financial theories, such as NPV, out of which better measurements and reasoning about this form of debt can take place in the wider context of software evolution or software improvement.

ABOUT THE AUTHORS

PHILIPPE KRUCHTEN is professor of software engineering at the University of British Columbia in Vancouver, Canada. A founding member of IFIP WG2.10, he conducts research in the software development process and software architecture. Kruchten received his PhD from the École Nationale Supérieure des Télécommunications in Paris. He's a professional engineer in Canada, an IEEE CSDP, and a senior (but not senile) member of the IEEE Computer Society. Contact him at kruchten@ieee.org.

ROBERT L. NORD is a senior member of the technical staff in the Research, Technology, and System Solutions Program at the Software Engineering Institute. He's engaged in activities focusing on agile and architecture at scale and works to develop and communicate effective methods and practices for software architecture. Nord received a PhD in computer science from Carnegie Mellon University and is a distinguished member of ACM. Contact him at rn@sei.cmu.edu.

IPEK OZKAYA is a senior member of the technical staff in the Research, Technology, and System Solutions Program at the Software Engineering Institute. She conducts research in empirical methods for improving software development efficiency and system evolution with a focus on software architecture practices, software economics, and requirements management. Ozkaya received a PhD in computational design from Carnegie Mellon University. She serves on the advisory board of *IEEE Software*. Contact her at ozkaya@sei.cmu.edu.

Acknowledgments

Many thanks to all the participants of the 3rd International Workshop on Technical Debt at ICSE 2012 in Zürich and to our reviewers, Len Bass and Raghvinder Sangwan. This material is based upon work funded and supported by the US Department of Defense under contract number FA8721-05-C-0003 with Carnegie Mellon University for the operation of the Software Engineering Institute, a federally funded research and development center. This material has been approved for public release and unlimited distribution.

References

1. W. Cunningham, "The WyCash Portfolio Management System," *Proc. OOPSLA*, ACM, 1992; http://c2.com/doc/oopsla92.html.
2. S. McConnell, "Technical Debt," blog, 2007; http://blogs.construx.com/blogs/stevemcc/archive/2007/11/01/technical-debt-2.aspx.
3. M. Fowler, "Technical Debt," blog, 2009; http://martinfowler.com/bliki/TechnicalDebt.html.
4. I. Gat, ed., "Special Issue on Technical Debt," *Cutter IT J.*, vol. 23, no. 10, 2010.
5. C. Sterling, *Managing Software Debt: Building for Inevitable Change*, Addison-Wesley Professional, 2010.
6. N. Brown et al., "Managing Technical Debt in Software-Intensive Systems," *Proc. Future of Software Eng. Research*, ACM, 2010, pp. 47–52; doi: 10.1145/1882362.1882373.
7. N. Brown, R. Nord, and I. Ozkaya, "Enabling Agility through Architecture," *CrossTalk*, Nov./Dec. 2010, pp. 12–18.
8. P. Kruchten, "What Colour Is Your Backlog?," blog, 2008; http://philippe.kruchten.com/talks.
9. P. Kruchten et al., "Report on the 3rd Workshop on Managing Technical Debt," to be published in *ACM SIGSOFT Software Eng. Notes*, vol. 37, no. 5, 2012; http:www.sigsoft.org/SEN.
10. M. Denne and J. Cleland-Huang, "The Incremental Funding Method: Data-Driven Software Development," *IEEE Software*, vol. 21, no. 3, 2004, pp. 39–47.

Selected CS articles and columns are also available for free at http://ComputingNow.computer.org.